FADENA

LICENCIATURA EN CIBERDEFENSA

DISPOSITIVOS REMOTOS E INTERNET DE LAS COSAS
CÁTEDRA: DIEGO RACERO
AÑO: 2026

CARGA HORARIA: 124 HS

EL VIAJE DEL DATO

Transformando información en decisiones estratégicas efectivas.

ETAPAS DEL VIAJE DEL DATO

El viaje del dato en sistemas IoT y embebidos representa el flujo completo desde la captura de información física hasta su transformación en conocimiento accionable. A continuación, encontrarás explicaciones detalladas de cada etapa:

PERCEPCIÓN CON SENSORES

Fundamentos de la Captura de Datos

Los sensores constituyen la interfaz primaria entre el mundo físico y el mundo digital. Son transductores que convierten magnitudes físicas (temperatura, presión, luz, movimiento, humedad, gases, etc.) en señales eléctricas medibles y procesables.

Tipos principales de sensores:

- **Sensores analógicos**: Generan señales continuas proporcionales a la magnitud medida (termistores, fotoresistores, potenciómetros)
- **Sensores digitales**: Proporcionan salidas discretas o comunicación mediante protocolos específicos (DHT22, BME280, acelerómetros digitales)
- **Sensores activos**: Requieren alimentación externa para operar (sensores ultrasónicos, infrarrojos activos)
- **Sensores pasivos**: Generan su propia señal sin necesidad de alimentación (termopares, células solares)

Características críticas de los sensores:

- **Rango de medición**: Valores mínimo y máximo que puede detectar
- **Resolución**: Cambio mínimo detectable en la magnitud medida
- **Precisión**: Cercanía entre el valor medido y el valor real
- **Exactitud**: Capacidad de reproducir la misma medición bajo condiciones idénticas
- **Tiempo de respuesta**: Velocidad con la que el sensor reacciona a cambios
- **Consumo energético**: Crucial en aplicaciones alimentadas por batería

Acondicionamiento de señales:

Antes de la digitalización, las señales analógicas frecuentemente requieren acondicionamiento mediante amplificadores operacionales, filtros pasa-bajos para eliminar ruido, divisores de voltaje para ajustar niveles, y conversores ADC (Analógico-Digital) que transforman la señal continua en valores discretos que el microcontrolador puede procesar.

INTERPRETACIÓN Y PROCESAMIENTO

El Cerebro del Sistema: El Microcontrolador

El microcontrolador actúa como el centro de procesamiento que recibe, interpreta y actúa sobre los datos capturados por los sensores.

Proceso de interpretación:

Lectura de datos brutos: El microcontrolador lee valores digitales provenientes de conversores ADC o directamente de sensores digitales. Por ejemplo, un ADC de 10 bits en un ESP32 proporciona valores entre 0 y 1023 que representan voltajes entre 0V y 3.3V.

Conversión y calibración: Los valores brutos se transforman mediante fórmulas matemáticas a unidades físicas significativas. Un sensor de temperatura LM35 que entrega 10mV por grado Celsius requiere la conversión: Temperatura_°C = (Lectura_ADC × 3.3V / 1024) / 0.01V.

Filtrado digital: Se aplican algoritmos como promedios móviles, filtros de Kalman o medianas para reducir ruido y fluctuaciones espurias en las lecturas.

Validación de datos: El microcontrolador verifica que las lecturas estén dentro de rangos esperados, descartando valores anómalos que podrían indicar fallas del sensor o interferencias.

Lógica de decisión:

El microcontrolador implementa algoritmos que determinan acciones basadas en los datos procesados. Esto puede incluir sistemas de control PID para mantener variables en setpoints deseados, máquinas de estados finitos que cambian comportamientos según condiciones, sistemas de alarma que activan alertas cuando se superan umbrales críticos, y control de actuadores como motores, relés, LEDs o pantallas.

Gestión de recursos:

Los microcontroladores operan con recursos limitados de memoria RAM, flash y capacidad de procesamiento. La programación eficiente mediante gestión cuidadosa de memoria dinámica, optimización de ciclos de reloj, implementación de modos de bajo consumo durante períodos de inactividad, y uso de interrupciones en lugar de polling continuo son esenciales para sistemas confiables y eficientes energéticamente.

CONEXIÓN INTERNA DE COMPONENTES

Protocolos de Comunicación Interna

La comunicación entre el microcontrolador y periféricos (sensores, actuadores, memoria externa, pantallas) utiliza diversos protocolos, cada uno optimizado para diferentes escenarios.

GPIO (General Purpose Input/Output):

GPIO representa las líneas más simples de comunicación. Cada pin puede configurarse como entrada o salida, con dos estados lógicos principales: HIGH (típicamente 3.3V o 5V) y LOW (0V). Son ideales para control de LEDs, lectura de botones, activación de relés, y generación de PWM (Pulse Width Modulation) para control de servomotores o regulación de intensidad lumínica.

Ventajas de GPIO: Simplicidad extrema, no requiere librerías especiales, velocidad máxima, control directo del hardware.

Limitaciones: Cada periférico requiere al menos un pin dedicado, no hay verificación de errores incorporada, alcance limitado a distancias muy cortas dentro del PCB.

I²C (Inter-Integrated Circuit):

I²C es un protocolo maestro-esclavo que permite conectar múltiples dispositivos (hasta 128 teóricamente) usando solo dos líneas: SDA (Serial Data) para transmisión bidireccional de datos y SCL (Serial Clock) para sincronización temporal.

Funcionamiento de I²C: El maestro (microcontrolador) inicia la comunicación enviando una condición de START, seguida por la dirección del esclavo (7 u 8 bits) con un bit R/W indicando lectura o escritura. El esclavo reconoce con un bit ACK (acknowledge). Los datos se transmiten byte por byte, cada uno seguido de un ACK del receptor. Finalmente, se envía una condición de STOP.

Ventajas: Uso eficiente de pines (solo dos sin importar cuántos dispositivos), direccionamiento permite identificación única de cada periférico, soporte para múltiples maestros en configuraciones avanzadas.

Limitaciones: Velocidad moderada (típicamente 100kHz en modo estándar, 400kHz en modo rápido), distancias cortas debido a capacitancia del bus, conflictos potenciales si dos dispositivos tienen la misma dirección.

Aplicaciones típicas: Sensores ambientales (BME280, BMP180), displays OLED y LCD, memorias EEPROM, relojes en tiempo real (RTC), expansores de puertos.

SPI (Serial Peripheral Interface):

SPI es un protocolo síncrono de alta velocidad que opera en modo maestro-esclavo, utilizando cuatro líneas principales: MOSI (Master Out Slave In) para envío de datos desde el maestro, MISO (Master In Slave Out) para recepción de datos, SCLK (Serial Clock) para sincronización, y SS/CS (Slave Select/Chip Select) para seleccionar el dispositivo activo.

Funcionamiento de SPI: El maestro activa la línea CS del esclavo deseado (típicamente llevándola a LOW), genera pulsos de reloj en SCLK, y transmite datos bit por bit en MOSI mientras simultáneamente recibe datos en MISO. La comunicación es full-duplex, permitiendo transmisión y recepción simultáneas.

Ventajas: Velocidades muy altas (varios MHz son comunes, algunos dispositivos soportan decenas de MHz), implementación sencilla sin protocolo complejo de direccionamiento, comunicación full-duplex, no hay limitación teórica en el número de dispositivos.

Limitaciones: Requiere una línea CS adicional por cada esclavo, consume más pines que I²C, no hay mecanismo de acknowledge incorporado, distancias limitadas por degradación de señal.

Aplicaciones típicas: Tarjetas SD, displays TFT, módulos de radiofrecuencia (nRF24L01), sensores de alta velocidad, convertidores ADC/DAC externos.

UART (Universal Asynchronous Receiver-Transmitter):

UART permite comunicación serial asíncrona punto a punto entre dos dispositivos usando solo dos líneas: TX (Transmit) para envío de datos y RX (Receive) para recepción. Las líneas se conectan cruzadas: TX de un dispositivo a RX del otro.

Funcionamiento de UART: No hay señal de reloj compartida; ambos dispositivos deben configurarse previamente con la misma velocidad de transmisión (baud rate como 9600, 115200 bps). Cada byte se transmite con un bit de START, 8 bits de datos (configurables), un bit opcional de paridad para detección de errores, y uno o dos bits de STOP.

Ventajas: Simplicidad, ampliamente soportado, ideal para comunicación con PC mediante adaptadores USB-Serial, no requiere sincronización compleja.

Limitaciones: Comunicación punto a punto únicamente, ambos dispositivos deben acordar previamente velocidad y formato, susceptible a errores si las velocidades no coinciden exactamente, sin direccionamiento de múltiples dispositivos.

Aplicaciones típicas: Comunicación con PC para debugging, módulos GPS, módulos Bluetooth y WiFi, impresoras, terminales de comandos.

CONTEXTUALIZACIÓN Y TELEMETRÍA

Transformación de Datos en Información Significativa

Los datos brutos capturados por sensores carecen de contexto útil hasta que se enriquecen con metadatos y se estructuran adecuadamente.

Elementos esenciales de la telemetría:

Timestamp (marca temporal): Cada lectura debe asociarse con el momento exacto de captura. Los timestamps pueden provenir de un RTC (Real-Time Clock) local, sincronización NTP (Network Time Protocol) para precisión, o contadores de tiempo relativo del microcontrolador (millis() en Arduino). El formato común es ISO 8601 o Unix timestamp (segundos desde 1970-01-01).

Identificación del dispositivo: En sistemas con múltiples sensores o nodos, cada mensaje debe incluir identificadores únicos como Device_ID o MAC address del dispositivo, ubicación geográfica o lógica (edificio, piso, sala), tipo de sensor o versión de firmware.

Metadatos de calidad: Indicadores que ayudan a evaluar la confiabilidad de los datos incluyen nivel de batería o voltaje de alimentación, intensidad de señal en comunicaciones inalámbricas (RSSI), número de secuencia para detectar pérdidas de paquetes, códigos de estado o error del sensor.

Formatos de datos estructurados:

JSON (JavaScript Object Notation) es el formato predominante en IoT por su legibilidad humana, amplio soporte en prácticamente todos los lenguajes de programación, y estructura flexible que admite objetos anidados y arrays.

Ejemplo de payload JSON completo:

json

```
{
  "device_id": "ESP32_001",
  "timestamp": "2026-01-03T14:35:22Z",
  "location": {
    "building": "A",
    "floor": 2,
    "room": "Lab_205"
  },
  "sensors": {
    "temperature": {
      "value": 23.5,
      "unit": "celsius"
    },
    "humidity": {
      "value": 58.2,
      "unit": "percent"
    },
    "motion": {
      "detected": true
    }
  },
  "device_status": {
    "battery_voltage": 3.7,
    "wifi_rssi": -67,
    "firmware_version": "1.2.3"
  }
}
```

Alternativas a JSON: XML ofrece mayor estructura y validación mediante esquemas pero es más verboso. CSV es eficiente para datos tabulares simples pero carece de jerarquía. Protocol Buffers y MessagePack proporcionan serialización binaria compacta para reducir ancho de banda en sistemas restringidos.

Compresión y optimización:

En dispositivos con memoria o ancho de banda limitados, se emplean técnicas como eliminación de espacios en blanco en JSON, envío de deltas (solo cambios desde la última transmisión), agregación temporal (enviar promedios o resúmenes de múltiples lecturas), y compresión de datos mediante algoritmos como gzip.

5. TRANSMISIÓN DE DATOS

Protocolos de Aplicación para Comunicación en Red

Una vez que los datos están contextualizados, deben transmitirse desde el dispositivo IoT hacia sistemas centrales para almacenamiento y análisis.

HTTP/HTTPS (Hypertext Transfer Protocol):

HTTP es el protocolo de aplicación más universal, operando sobre TCP/IP. Los dispositivos IoT típicamente actúan como clientes HTTP enviando datos mediante métodos POST o PUT hacia APIs REST.

Ventajas de HTTP: Universalmente soportado, atraviesa fácilmente firewalls corporativos, amplia disponibilidad de librerías, compatibilidad con estándares web existentes, soporte para autenticación mediante API keys o tokens OAuth.

Desventajas: Overhead significativo (headers HTTP agregan cientos de bytes por mensaje), establecimiento de conexión TCP costoso en términos de energía y latencia, inadecuado para dispositivos de ultra bajo consumo o con conexiones intermitentes.

Ejemplo de transmisión HTTP POST:
```
POST /api/v1/telemetry HTTP/1.1
Host: iot-server.example.com
Content-Type: application/json
Authorization: Bearer abc123xyz
Content-Length: 245

{payload JSON aquí}
```

MQTT (Message Queuing Telemetry Transport):

MQTT es un protocolo ligero de publicación-suscripción diseñado específicamente para IoT. Opera sobre TCP y utiliza un broker central que gestiona la distribución de mensajes entre publishers (publicadores) y subscribers (suscriptores).

Arquitectura MQTT: Los dispositivos se conectan al broker y publican mensajes en topics (temas jerárquicos como "edificio/piso2/temperatura"). Otros clientes suscritos a esos topics reciben automáticamente los mensajes. El broker gestiona la distribución sin que publicadores y suscriptores necesiten conocerse mutuamente.

Niveles de QoS (Quality of Service): QoS 0 (at most once) envía el mensaje sin confirmación, ideal para datos no críticos. QoS 1 (at least once) garantiza entrega pero puede duplicar mensajes. QoS 2 (exactly once) asegura entrega única mediante un handshake de cuatro vías, apropiado para comandos críticos.

Ventajas de MQTT: Overhead mínimo (headers de solo 2 bytes en casos óptimos), soporte nativo para dispositivos con conexiones intermitentes mediante sesiones persistentes, publicación-suscripción permite arquitecturas desacopladas y escalables, soporte para mensajes LWT (Last Will and Testament) que notifican desconexiones inesperadas.

CoAP (Constrained Application Protocol):

CoAP es un protocolo especializado para dispositivos altamente restringidos, operando sobre UDP en lugar de TCP para reducir overhead. Diseñado como un análogo ligero de HTTP para redes de sensores.

Características de CoAP: Utiliza métodos similares a HTTP (GET, POST, PUT, DELETE), emplea códigos de respuesta compatibles, soporta observación de recursos para notificaciones asíncronas, y permite multicast para comunicación uno-a-muchos eficiente.

Consideraciones de seguridad en transmisión:

La seguridad es crítica en IoT. HTTPS proporciona cifrado mediante TLS/SSL, protegiendo datos en tránsito. MQTT puede operar sobre TLS (MQTTS). La autenticación mediante certificados X.509, API keys o tokens JWT previene accesos no autorizados. Firmado digital de mensajes garantiza integridad y autenticidad.

RECEPCIÓN Y ANÁLISIS

Backend: Procesamiento, Almacenamiento y Generación de Insights

El servidor o plataforma IoT constituye el destino final del viaje del dato, donde se realiza procesamiento avanzado y se extrae valor de la información recopilada.

Recepción y validación inicial:

Cuando el servidor recibe datos, el primer paso es la validación rigurosa. Esto incluye verificación de formato (parsing JSON correcto), validación de esquema (presencia de campos obligatorios, tipos de datos correctos), autenticación del dispositivo emisor, verificación de integridad mediante checksums o firmas digitales, y detección de duplicados mediante números de secuencia o timestamps.

Almacenamiento de datos:

La elección de la base de datos depende de los requisitos del sistema:

Bases de datos de series temporales como InfluxDB, TimescaleDB o Prometheus están optimizadas para datos timestamped con escrituras secuenciales de alta frecuencia, consultas agregadas sobre ventanas temporales, retención automática basada en políticas, y compresión eficiente de datos históricos.

Bases de datos relacionales (PostgreSQL, MySQL) son apropiadas para metadatos estructurados, relaciones complejas entre entidades, y transacciones ACID cuando la consistencia es crítica.

Bases de datos NoSQL (MongoDB, Cassandra) ofrecen escalabilidad horizontal para volúmenes masivos, esquemas flexibles para datos heterogéneos, y alta disponibilidad mediante replicación.

Procesamiento y análisis:

Procesamiento en tiempo real (streaming): Sistemas como Apache Kafka, Apache Flink o AWS Kinesis procesan datos conforme llegan, permitiendo detección de anomalías inmediata, generación de alertas en milisegundos, agregaciones en ventanas temporales móviles, y transformaciones y enriquecimiento de datos en tránsito.

Análisis batch: Para insights históricos, se ejecutan análisis periódicos sobre conjuntos completos de datos mediante consultas SQL complejas, cálculo de estadísticas agregadas (promedios, máximos, mínimos, desviaciones estándar), identificación de tendencias y patrones temporales, y generación de reportes programados.

Machine Learning e IA: Los datos históricos alimentan modelos predictivos para mantenimiento predictivo (predecir fallas antes de que ocurran), optimización energética, detección de anomalías mediante algoritmos de clustering o redes neuronales, y reconocimiento de patrones complejos.

Visualización y dashboards:

Los insights se presentan mediante interfaces gráficas usando herramientas como Grafana, Kibana o dashboards personalizados. Las visualizaciones incluyen gráficos de series temporales mostrando evolución de variables, mapas de calor para distribuciones espaciales, indicadores KPI para métricas críticas, y alertas visuales cuando se superan umbrales.

Acción automatizada:

El análisis puede desencadenar acciones automáticas como envío de comandos de vuelta a dispositivos IoT (control remoto), notificaciones push, emails o SMS para alertas críticas, webhooks hacia sistemas externos, y actualización de sistemas SCADA o ERP empresariales.

ANEXO I (descripción de tallara de protocolos de comunicación interna)

PROTOCOLOS DE COMUNICACIÓN INTERNA: GPIO, I²C, SPI Y UART

GPIO (General Purpose Input/Output)

Conceptos Fundamentales

GPIO representa las líneas digitales más básicas y versátiles de un microcontrolador. Cada pin GPIO puede configurarse dinámicamente mediante software como entrada o salida, permitiendo interfaz directa con el mundo exterior.

Configuración como salida:

En modo salida, el microcontrolador controla el estado eléctrico del pin. Un estado HIGH típicamente representa voltaje positivo de alimentación (3.3V en ESP32, 5V en Arduino tradicional), mientras que LOW representa 0V (tierra). Este control permite activar LEDs, relés, transistores, displays de 7 segmentos, y cualquier dispositivo que requiera señales de control digital.

Corriente y protección: Cada pin GPIO tiene limitaciones de corriente (típicamente 20-40mA). Exceder estos límites daña permanentemente el microcontrolador. Para cargas mayores se utilizan transistores MOSFET o relés como interfaces de potencia.

PWM (Pulse Width Modulation): Aunque GPIO opera en estados binarios, PWM permite simular salidas analógicas variando la proporción de tiempo en HIGH vs LOW. Con una frecuencia suficientemente alta (>50Hz), el promedio de voltaje percibido cambia. Esto permite control de brillo LED, velocidad de motores DC, y generación de señales de control para servomotores.

Configuración como entrada:

En modo entrada, GPIO lee el estado eléctrico presente en el pin. Aplicaciones comunes incluyen lectura de botones y switches, sensores digitales simples, detectores de movimiento PIR, y encoders rotativos.

Resistencias pull-up y pull-down: Las entradas flotantes (no conectadas a HIGH ni LOW) son inestables y capturan ruido electromagnético. Las resistencias pull-up (típicamente 10kΩ conectadas a VCC) mantienen el pin en HIGH por defecto, mientras que pull-down lo mantienen en LOW. Muchos microcontroladores incluyen resistencias pull-up/down internas activables por software.

Interrupciones: En lugar de verificar continuamente el estado de un pin (polling), las interrupciones permiten ejecutar código automáticamente cuando ocurre un cambio. Los triggers comunes son RISING (transición LOW→HIGH), FALLING (HIGH→LOW), CHANGE (cualquier transición), y LOW/HIGH (nivel constante).

Ventajas de GPIO:

- Máxima simplicidad de implementación
- Velocidad de respuesta inmediata (nanosegundos)
- Control directo del hardware sin capas de abstracción
- Ideal para señales de control simples

Limitaciones de GPIO:

- Escalabilidad pobre: cada dispositivo consume un pin dedicado
- Sin protocolo de comunicación estructurado
- Alcance limitado a trazas del PCB
- No hay transmisión de datos complejos sin implementar protocolos personalizados

I²C (Inter-Integrated Circuit)

Arquitectura y Funcionamiento Profundo

I²C, desarrollado por Philips (ahora NXP), es un bus de comunicación serial síncrono multi-maestro/multi-esclavo que revolucionó la conexión de periféricos mediante su diseño de solo dos cables.

Líneas del bus:

SDA (Serial Data): Línea bidireccional para transmisión de datos, direcciones y reconocimientos. Tanto maestro como esclavo pueden controlarla en diferentes momentos.

SCL (Serial Clock): Generada por el maestro, sincroniza todas las transferencias. Los esclavos nunca generan clock en operación estándar, aunque pueden extender pulsos de clock mediante clock stretching.

Topología de drenaje abierto (open-drain): Ambas líneas utilizan configuración open-drain con resistencias pull-up externas (típicamente 4.7kΩ). Esto permite que múltiples dispositivos compartan el bus sin cortocircuitos: cualquier dispositivo puede llevar la línea a LOW, pero solo las resistencias pull-up la elevan a HIGH. Esta característica habilita arbitraje de bus en sistemas multi-maestro.

Protocolo de comunicación detallado:

Condición START: El maestro inicia comunicación generando una transición HIGH→LOW en SDA mientras SCL permanece HIGH. Esta condición única alerta a todos los dispositivos del bus que comienza una transacción.

Transmisión de dirección: Inmediatamente después de START, el maestro envía 7 bits de dirección del esclavo objetivo (I²C estándar) o 10 bits (I²C extendido para más de 128 dispositivos). El octavo bit es R/$\overline{W}$: 0 para escritura, 1 para lectura.

Bit de reconocimiento (ACK): Después de cada byte transmitido, el receptor debe generar un ACK llevando SDA a LOW durante el noveno pulso de clock. La ausencia de ACK (NACK) indica fin de transmisión o error.

Transmisión de datos: Los datos se envían byte por byte, MSB primero. Cada byte es seguido por ACK/NACK.

Condición STOP: Finaliza la transacción con una transición LOW→HIGH en SDA mientras SCL está en HIGH.

Direccionamiento I²C:

Los dispositivos I²C tienen direcciones de 7 bits asignadas por el fabricante, aunque algunos bits pueden ser configurables mediante pines de hardware. Direcciones comunes reservadas incluyen 0x00 (general call), 0x01-0x07 y 0x78-0x7F (reservadas). Esto deja aproximadamente 112 direcciones utilizables.

Conflictos de dirección: Cuando dos dispositivos comparten dirección, pueden usarse multiplexores I²C (como TCA9548A) que crean múltiples buses virtuales, o configurar direcciones alternativas si el dispositivo lo permite.

Velocidades de operación:

- **Modo estándar**: 100 kHz
- **Modo rápido**: 400 kHz
- **Modo rápido plus**: 1 MHz
- **Modo de alta velocidad**: 3.4 MHz

La velocidad práctica depende de la capacitancia total del bus (longitud de cables, número de dispositivos) y las capacidades de todos los dispositivos conectados.

Clock stretching: Mecanismo que permite a esclavos lentos pausar la comunicación manteniendo SCL en LOW después de recibir un byte, dando tiempo para procesar datos antes de continuar.

Ventajas de I²C:

- Eficiencia extrema de pines: solo dos líneas para múltiples dispositivos
- Direccionamiento automático elimina selección manual
- Protocolo estandarizado con amplio soporte de librerías
- Permite lectura directa de registros internos de dispositivos complejos

Limitaciones de I²C:

- Velocidad moderada comparada con SPI
- Capacitancia limita longitud de cables (típicamente <1 metro sin extenders)

- Conflictos de dirección requieren gestión cuidadosa
- Susceptibilidad a ruido electromagnético en cables largos
- Requiere resistencias pull-up externas adecuadas

Aplicaciones ideales:

Sensores ambientales de baja velocidad, displays OLED y LCDs I²C, memorias EEPROM, conversores ADC externos de precisión, expansores de GPIO (PCF8574), relojes RTC, y giroscopios/acelerómetros.

SPI (Serial Peripheral Interface)

Comunicación de Alta Velocidad

SPI es un protocolo de comunicación serial síncrono full-duplex desarrollado por Motorola, diseñado específicamente para velocidades de transferencia muy altas en comunicación de corta distancia.

Líneas del bus SPI:

SCLK (Serial Clock): Generada exclusivamente por el maestro, sincroniza todas las transferencias. La velocidad puede alcanzar decenas de MHz dependiendo del dispositivo.

MOSI (Master Out Slave In): Línea de datos del maestro hacia el esclavo. El maestro transmite y el esclavo recibe.

MISO (Master In Slave Out): Línea de datos del esclavo hacia el maestro. Permite comunicación full-duplex: transmisión y recepción simultáneas.

SS/CS (Slave Select / Chip Select): Línea individual por cada esclavo que activa/desactiva su participación en la comunicación. Típicamente activa en LOW (lógica negativa).

Funcionamiento del protocolo:

SPI no tiene estructura de frames o paquetes predefinidos como I²C. El protocolo es extremadamente simple:

1. El maestro activa CS del esclavo deseado (LOW)
2. El maestro genera pulsos de clock en SCLK
3. En cada flanco de clock, se transfiere simultáneamente un bit en MOSI (maestro→esclavo) y un bit en MISO (esclavo→maestro)
4. Después de transferir el número deseado de bits/bytes, el maestro desactiva CS (HIGH)

Modos de operación SPI:

SPI define cuatro modos basados en dos parámetros:

CPOL (Clock Polarity): Define el estado inactivo del clock

- CPOL=0: Clock inactivo en LOW
- CPOL=1: Clock inactivo en HIGH

CPHA (Clock Phase): Define en qué flanco se capturan datos

- CPHA=0: Datos muestreados en el primer flanco de clock
- CPHA=1: Datos muestreados en el segundo flanco de clock

Los cuatro modos son:

- **Modo 0** (CPOL=0, CPHA=0): Clock inactivo LOW, muestreo en flanco ascendente
- **Modo 1** (CPOL=0, CPHA=1): Clock inactivo LOW, muestreo en flanco descendente
- **Modo 2** (CPOL=1, CPHA=0): Clock inactivo HIGH, muestreo en flanco descendente
- **Modo 3** (CPOL=1, CPHA=1): Clock inactivo HIGH, muestreo en flanco ascendente

Maestro y esclavo deben configurarse en el mismo modo para comunicación exitosa.

Configuración multi-esclavo:

Arquitectura estándar: Cada esclavo tiene su propia línea CS pero comparte SCLK, MOSI y MISO. El maestro activa solo un CS a la vez. Esto requiere N+3 pines para N esclavos.

Daisy chain: Los esclavos se conectan en serie, donde MISO de uno conecta a MOSI del siguiente. Todos comparten CS. Reduce pines pero requiere soporte específico del hardware.

Ventajas de SPI:

- Velocidades extremadamente altas (10-100 MHz comunes)
- Comunicación full-duplex simultánea
- No hay overhead de direccionamiento: activación directa vía CS
- Protocolo simple sin complejidad de arbitraje
- Menos susceptible a ruido que I²C debido a señales más rápidas
- No requiere resistencias pull-up

Limitaciones de SPI:

- Consumo alto de pines: 4 + (N-1) pines para N esclavos
- Sin mecanismo de acknowledge: no hay confirmación de recepción
- Distancias muy limitadas (centímetros a ~1 metro máximo)
- Sin direccionamiento automático: debe gestionar CS manualmente
- Sin soporte multi-maestro estándar

Aplicaciones ideales:

Tarjetas SD y memoria flash, displays TFT de alta resolución, módulos de comunicación inalámbrica (nRF24L01, LoRa), conversores ADC/DAC de alta velocidad, sensores de imagen, y acelerómetros/giroscopios de alta frecuencia de muestreo.

UART (Universal Asynchronous Receiver-Transmitter)

Comunicación Serial Asíncrona

UART es uno de los protocolos de comunicación más antiguos y ubicuos, utilizado extensivamente para comunicación punto a punto confiable sin necesidad de señal de clock compartida.

Principio de operación asíncrona:

A diferencia de protocolos síncronos (I²C, SPI), UART no transmite señal de clock. En su lugar, ambos dispositivos deben preconfigurarse con parámetros idénticos de comunicación antes de conectarse.

Líneas UART:

TX (Transmit): Salida de datos del dispositivo **RX (Receive)**: Entrada de datos al dispositivo **Conexión cruzada**: TX de un dispositivo conecta a RX del otro, y viceversa **GND**: Referencia común de voltaje (crítico para comunicación confiable)

Opcionalmente puede incluir líneas de control de flujo hardware: RTS (Request To Send) y CTS (Clear To Send) para coordinar transmisión.

Estructura del frame UART:

Cada byte se transmite en un frame que incluye:

Bit de START: Siempre 0 (LOW), señala inicio de transmisión. La línea está normalmente en estado HIGH (idle), por lo que la transición HIGH→LOW alerta al receptor.

Bits de datos: Típicamente 8 bits transmitidos LSB primero, aunque configuraciones de 5, 6, 7 o 9 bits son posibles.

Bit de paridad (opcional): Permite detección básica de errores. Paridad par (even) hace que el número total de bits en 1 sea par. Paridad impar (odd) hace que sea impar. Ninguna paridad omite este bit.

Bits de STOP: Uno o dos bits en HIGH señalan fin de transmisión y proporcionan separación temporal antes del siguiente frame.

Parámetros de configuración críticos:

Baud rate: Velocidad de transmisión en bits por segundo. Valores comunes: 9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600 bps. Ambos dispositivos deben usar exactamente el mismo baud rate.

Bits de datos: Típicamente 8 bits (1 byte)

Paridad: None, Even, Odd, Mark (siempre 1), Space (siempre 0)

Bits de stop: 1 o 2 bits

La configuración se especifica frecuentemente como "8N1" (8 bits de datos, sin paridad, 1 bit de stop) o "8E2" (8 bits, paridad par, 2 bits de stop).

Timing y sincronización:

El receptor debe muestrear la línea RX a la velocidad correcta para interpretar bits. Cuando detecta el bit de START (HIGH→LOW), inicia un timer que muestrea cada bit en el centro de su ventana temporal. Desviaciones en el baud rate superiores al ~5% causan errores de lectura.

Osciladores de cristal de precisión son críticos para comunicación UART confiable a altas velocidades.

Control de flujo:

Hardware flow control: Usa líneas RTS/CTS. El receptor baja CTS cuando su buffer está lleno, señalando al transmisor que pause.

Software flow control (XON/XOFF): Usa caracteres especiales (CTRL+S / CTRL+Q) transmitidos en el propio stream de datos para pausar/reanudar.

Ventajas de UART:

- Simplicidad extrema: solo dos cables más tierra
- Ampliamente soportado en prácticamente todo hardware
- Comunicación confiable con detección de errores opcional
- Ideal para comunicación con PC mediante adaptadores USB-Serial
- Comunicación full-duplex independiente
- No requiere configuración de direcciones

Limitaciones de UART:

- Comunicación estrictamente punto a punto (un transmisor, un receptor)
- Ambos extremos deben preconfigurar exactamente los mismos parámetros
- Velocidades moderadas comparadas con SPI
- Susceptible a errores si los clocks no están bien sincronizados
- Distancias limitadas sin conversión a estándares como RS-232 o RS-485

Niveles de voltaje:

UART lógico: Opera a niveles TTL/CMOS (0V y 3.3V/5V)

RS-232: Estándar de señalización que usa voltajes +/-3V a +/-15V para mayor inmunidad a ruido y mayores distancias (hasta 15 metros). Requiere conversores de nivel como MAX3232.

RS-485: Señalización diferencial para distancias de hasta 1200 metros y ambientes industriales ruidosos, con soporte multi-drop para varios dispositivos en bus.

Aplicaciones ideales:

Comunicación con PC para debugging y terminales, módulos GPS que transmiten sentencias NMEA, módulos Bluetooth HC-05/HC-06, módulos WiFi ESP8266, lectores RFID, impresoras, y comunicación entre microcontroladores.

Este conjunto de protocolos forma el ecosistema fundamental de comunicación en sistemas embebidos e IoT. La elección del protocolo apropiado depende de factores como velocidad requerida, número de dispositivos, distancia, disponibilidad de pines, consumo energético, y complejidad de implementación. Frecuentemente, un sistema complejo utiliza múltiples protocolos simultáneamente, cada uno optimizado para su tarea específica.